Datathon Revision Notes: Module 1 Linear Algebra Foundations

Roman

May 21, 2026

Core Architecture: The Data Hierarchy

The Inventor's Problem

How do we bundle massive collections of multi-featured data into a single, clean mathematical object that a computer can process all at once, avoiding chaotic, slow nested loops?

The Analogy

Think of data dimensionality as structural building blocks:

- **0D Scalar (α):** A single configuration value floating in isolation (e.g., your model's learning rate like $\alpha = 0.01$).
- **1D Vector (x):** A single data profile or customer passport. It is a straight line of numbers where each position represents a specific feature (e.g., a customer's [Age, Income, Credit Score]).
- **2D Matrix (X):** An entire spreadsheet grid layout. It stacks multiple customer vectors on top of each other. Rows represent individual samples; columns represent specific features.
- **ND Tensor:** Higher-dimensional data shapes (e.g., a 3D tensor stacking multiple 2D matrices over time, like tracking customer spreadsheets month-by-month).

The First-Principles Logic

By organizing features into contiguous matrices, we unlock hardware-level optimizations. Instead of using Python `for` loops to inspect each row sequentially, modern CPUs and GPUs use SIMD (Single Instruction, Multiple Data) architectures to run vector operations simultaneously across an entire grid in a single clock cycle.

Alignment and Measurement: Transpose & Dot Product

The Inventor's Problem

How do we manipulate the spatial orientation of an incompatible data grid to satisfy matching constraints, and how do we compute a single similarity score showing how well two multi-featured variables align or push together?

The Analogy

- **Matrix Transpose ($\mathbf{X}^T$):** Think of this as a physical orientation adapter plug. If your matrix is standing up tall (Rows $\times$ Columns) and you need it to lay flat sideways to plug into a multiplication formula, the transpose flips it across its diagonal axis, turning all rows into columns and columns into rows.
- **Vector Dot Product ($\mathbf{u} \cdot \mathbf{v}$):** Think of this as a **Directional Alignment Meter**. If two vectors are pointing in the exact same direction, their dot product explodes to its maximum value. If they point at a right angle (90°), they share zero alignment and their dot product is exactly 0.

The First-Principles Logic

The core engine of a machine learning prediction layer is a dot product between a feature vector and a learned weight vector:

$$\hat{y} = \mathbf{w}^T\mathbf{x} + b = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

The model evaluates incoming customer features $\mathbf{x}$ against its internalized weight pattern $\mathbf{w}$. The stronger the structural alignment (higher dot product), the higher the output probability score.

Single-Step Optimization: The Normal Equation

The Inventor's Problem

When dealing with an overdetermined system of equations (vastly more rows/samples than columns/features), a perfect zero-error solution is mathematically impossible. How do we compute the single best possible weight vector that minimizes total squared error across every row simultaneously in one step?

The Analogy

Imagine a scatterplot with millions of chaotic data points. You are holding a straight wooden stick, trying to drop it into the data cloud so that it acts as a perfect average centerline. You cannot bend the stick to touch every single point. The Normal Equation calculates the exact structural equilibrium point where the gravitational pull of all points combined forces the stick into the absolute minimum total squared error position.

The First-Principles Logic

We start with the ideal, unresolvable linear system: $\mathbf{Xw} = \mathbf{y}$. Because the rectangular matrix $\mathbf{X}$ is not square, we cannot compute its inverse directly. To resolve this, we multiply both sides of the equation by the adapter plug $\mathbf{X}^T$:

$$\mathbf{X}^T\mathbf{Xw} = \mathbf{X}^T\mathbf{y}$$

The multiplication $\mathbf{X}^T\mathbf{X}$ collapses the tall rectangular data matrix into a neat, symmetric square matrix. We then isolate the optimal weight vector $\mathbf{w}$ by multiplying by the computed square inverse:

$$\mathbf{w} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

Dimensionality Compression: Eigenvalues & PCA

The Inventor's Problem

How do we look at a massive, redundant cloud of data points containing hundreds of highly correlated columns and isolate the absolute most informative directional axes so we can compress the space without losing critical predictive signal?

The Analogy

Imagine looking at a complex, three-dimensional wire sculpture hanging in a room. If you shine a flashlight on it from a random angle, its shadow on the flat wall becomes a chaotic, unreadable smear. But if you slowly walk around the sculpture until you find the perfect perspective, the shadow suddenly projects a clean, wide silhouette that captures the entire structure of the object. Principal Component Analysis (PCA) finds that exact optimal perspective angle.

The First-Principles Logic

An eigenvector ($\mathbf{v}$) represents a special directional axis in your feature space that does not change its spatial orientation when a transformation matrix $\mathbf{A}$ is applied to it—it only scales. The eigenvalue (λ) is the scale factor that quantifies exactly how much data variance is distributed along that axis:

$$\mathbf{Av} = \lambda\mathbf{v}$$

By calculating the eigenvectors of a dataset's covariance matrix, PCA identifies the axes of maximum variance. Projecting your high-dimensional data onto the top-scoring eigenvectors allows you to compress wide data tables cleanly while filtering out random noise.

Overfitting Protection: Vector Norms (L_1 & L_2)

The Inventor's Problem

How do we mathematically track and punish the physical magnitude of a model's weight vector to prevent it from using massive, volatile coefficients to memorize random training noise?

The Analogy

- L_1 **Regularization (Manhattan Distance / Taxi Tax):** This acts like a strict flat-rate toll collector. It looks at the absolute sum of the weights ($\sum |w_i|$). It doesn't care if a weight is small or large; it applies a steady, constant downward pressure. This pressure forces less critical weights to drop all the way down to exactly 0, acting as an automatic feature selection engine.
- L_2 **Regularization (Euclidean Distance / Crow-Flies Tax):** This acts like an emergency siren. It squares the weights ($\sum w_i^2$). If a weight is tiny (e.g., 0.1), its squared penalty is microscopic (0.01). But if a weight balloons out of control to +90.0, the penalty skyrockets exponentially (8100.0). This aggressively crushes massive outlier parameters, forcing the model to distribute its attention evenly across all features.